

Respuestas a las Preguntas Conceptuales

1. Sobre Observables y Asincronía

a. ¿Por qué las peticiones HTTP en Angular devuelven un Observable en lugar de la data directa? ¿Qué diferencia hay frente a una Promise?

Las peticiones HTTP devuelven Observables porque representan flujos de datos que pueden emitir múltiples valores a lo largo del tiempo, aunque en el caso de HTTP típicamente emiten solo uno. La principal diferencia con las Promises es que los Observables son "lazy", es decir, no se ejecutan hasta que alguien se suscribe a ellos, mientras que una Promise se ejecuta inmediatamente al crearse. Además, los Observables permiten cancelar la suscripción si ya no necesito los datos, algo imposible con Promises. También puedo encadenar operadores RxJS como map, filter o retry para transformar y controlar el flujo de datos de manera más elegante. Esta arquitectura reactiva me da mayor control sobre el manejo asíncrono.

b. ¿Qué ocurre si un componente llama a un servicio HTTP pero nunca hace .subscribe()? ¿Se ejecuta la petición igualmente?

No, la petición HTTP no se ejecuta en absoluto. Esto es lo que significa que los Observables sean "lazy" o perezosos: el código dentro del Observable no se ejecuta hasta que alguien se suscribe explícitamente. Puedo declarar `this.cityService.getCities()` mil veces, pero si nunca agrego `.subscribe()`, Angular simplemente no hará la llamada al servidor. Es como tener una receta de cocina en mis manos pero nunca cocinar realmente el plato. Esta característica es útil porque me permite definir flujos de datos complejos sin que se ejecuten hasta que realmente los necesite, ahorrando recursos y dándome control sobre cuándo iniciar las operaciones asíncronas.

2. Sobre Inyección de Dependencias

c. ¿Qué ventaja de modularidad da `providedIn: 'root'` frente a instanciar el servicio manualmente con `new CityService()` dentro de un componente?

La ventaja principal es que Angular crea una única instancia del servicio (singleton) para toda la aplicación, compartida entre todos los componentes que la necesiten. Si yo usara `new CityService()` en cada componente, tendría múltiples instancias independientes, lo que consumiría más memoria y haría imposible compartir estado entre componentes. Además, `providedIn: 'root'` permite tree-shaking: si nunca uso el servicio en mi aplicación, Angular lo elimina del bundle final, optimizando el tamaño. También me facilita el testing porque puedo reemplazar fácilmente el servicio real por un mock sin tocar el código del componente. La inyección automática hace mi código más limpio y mantenible.

d. ¿Cómo facilitaría la inyección de dependencias la escritura de pruebas unitarias para estos servicios?

La inyección de dependencias hace que escribir tests sea mucho más sencillo porque puedo reemplazar las dependencias reales por mocks o stubs sin modificar el código de producción. Por ejemplo, cuando quiero probar `CityDetailComponent`, no necesito que realmente llame a `WeatherAPI`; simplemente

inyecto un WeatherService falso que devuelva datos predefinidos. Esto me permite aislar la lógica que estoy probando, hacer los tests más rápidos (sin llamadas HTTP reales) y más confiables (sin depender de servicios externos que puedan fallar). Angular provee utilidades como TestBed que facilitan esta configuración, permitiéndome especificar qué implementación quiero inyectar en cada test.

3. Sobre Interceptores

e. ¿Por qué es mejor centralizar el manejo de errores en `HttpInterceptor` en lugar de poner bloques `try/catch` en cada componente?

Centralizar el manejo de errores en un interceptor elimina la duplicación de código y garantiza consistencia en toda la aplicación. Si pongo `try/catch` en cada componente, terminaría repitiendo la misma lógica de mostrar notificaciones Toastr en 20 lugares diferentes, y si luego quiero cambiar el formato del mensaje de error, tendría que modificar todos esos archivos. Con el interceptor, tengo un único punto donde defino cómo se manejan los errores HTTP, lo que facilita el mantenimiento. Además, puedo implementar lógica transversal como logging automático, reintentos, o agregar headers de autenticación sin que los componentes sepan nada al respecto. Esto separa responsabilidades: los componentes se enfocan en la lógica de negocio y el interceptor en aspectos técnicos.

f. Además del manejo de errores, mencione dos casos de uso adicionales para un interceptor.

Un caso de uso común es agregar automáticamente tokens de autenticación a todas las peticiones HTTP salientes. En lugar de añadir manualmente el header `Authorization: Bearer <token>` en cada llamada al servicio, el interceptor puede interceptar todas las requests y agregarle el token desde un servicio de autenticación. Otro caso útil es implementar un sistema de logging centralizado que registre todas las peticiones y respuestas HTTP para debugging o auditoría, capturando información como la URL llamada, el tiempo de respuesta y el status code. También podría usar interceptores para mostrar/ocultar un spinner de carga global, transformar respuestas, implementar caché, o agregar timestamps a las requests.

4. Sobre el Patrón Maestro-Detalle

g. ¿Por qué se usa `@Input()` para pasar la ciudad a `CityDetailComponent` en lugar de volver a hacer un GET a `/api/cities/{id}`?

Usar `@Input()` es más eficiente porque la información ya está disponible en el componente padre cuando el usuario hace clic en "Ver Detalle". Si hiciera otro GET, estaría haciendo una llamada innecesaria al servidor para obtener datos que ya tengo en memoria, lo que añade latencia, consume ancho de banda y genera carga adicional en el backend. Además, con `@Input()` mantengo una arquitectura más clara donde el componente hijo es "tonto" (presentational): solo recibe datos y los muestra, sin preocuparse de dónde vienen. Esto hace el componente más reutilizable y fácil de testear. La única vez que justificaría un GET sería si necesitara datos frescos o información adicional que no viene en la lista inicial.

h. Observe que en la respuesta de GET /api/cities, el campo country ya viene como objeto anidado (no solo el countryId). ¿Qué ventaja de diseño tiene esta decisión del backend para el frontend?

Esta decisión evita el problema de "N+1 queries" en el frontend y simplifica enormemente el código. Si el backend solo devolviera countryId, tendría que hacer una llamada adicional a /api/countries/{id} por cada ciudad para obtener el nombre del país y mostrarlo en la tabla. Con el objeto anidado, una sola petición me da toda la información necesaria. Esto reduce la cantidad de requests HTTP, mejora el performance de la aplicación y hace el código del frontend más simple porque no necesito coordinar múltiples llamadas asíncronas ni manejar estados de carga complejos. Es un ejemplo de cómo un buen diseño de API facilita el desarrollo frontend.